

Lenguajes y Compiladores — Apunte para el Final
Resumen de Reynolds, *Theories of Programming Languages* (2009)

Nicolás Cagliero

28 de junio de 2026

Índice general

1. Dominios, punto fijo y lenguaje imperativo	1
1.1. El lenguaje imperativo simple (Reynolds §2.1–2.2)	1
1.2. Dominios y funciones continuas (Reynolds §2.3)	3
2. Cálculo lambda	7
3. Lenguajes aplicativos e Iswim	8

Capítulo 1

Dominios, punto fijo y lenguaje imperativo

1.1. El lenguaje imperativo simple (Reynolds §2.1–2.2)

Motivación

Queremos darle un *significado* matemático preciso a un lenguaje imperativo. Las cuatro primeras construcciones salen directo; el **while** es el que obliga a inventar toda la maquinaria de dominios y punto fijo (§2.3–2.4). Este capítulo deja planteado ese problema.

Sintaxis

$$\begin{aligned}\langle \textit{intexp} \rangle &::= 0 \mid 1 \mid 2 \mid \dots \mid \langle \textit{var} \rangle \mid -\langle \textit{intexp} \rangle \mid \langle \textit{intexp} \rangle + \langle \textit{intexp} \rangle \mid \dots \mid \langle \textit{intexp} \rangle \mathbf{rem} \langle \textit{intexp} \rangle \\ \langle \textit{boolexp} \rangle &::= \mathbf{true} \mid \mathbf{false} \mid \langle \textit{intexp} \rangle = \langle \textit{intexp} \rangle \mid \langle \textit{intexp} \rangle < \langle \textit{intexp} \rangle \mid \dots \mid \neg \langle \textit{boolexp} \rangle \mid \langle \textit{boolexp} \rangle \wedge \langle \textit{boolexp} \rangle \\ \langle \textit{comm} \rangle &::= \langle \textit{var} \rangle := \langle \textit{intexp} \rangle \mid \mathbf{skip} \mid \langle \textit{comm} \rangle ; \langle \textit{comm} \rangle \\ &\quad \mid \mathbf{if} \langle \textit{boolexp} \rangle \mathbf{then} \langle \textit{comm} \rangle \mathbf{else} \langle \textit{comm} \rangle \mid \mathbf{while} \langle \textit{boolexp} \rangle \mathbf{do} \langle \textit{comm} \rangle\end{aligned}$$

Todas las variables son enteras (no hay variables booleanas).

Estados

Definición 1.1 (Estado y actualización). Un *estado* $\sigma \in \Sigma$ asigna a cada variable un entero. La *actualización* $[\sigma \mid v : n]$ es el estado igual a σ salvo que manda $v \mapsto n$:

$$[\sigma \mid v : n](w) = \begin{cases} n & \text{si } w = v \\ \sigma(w) & \text{si } w \neq v. \end{cases}$$

Semántica de expresiones

$$\llbracket \cdot \rrbracket_{\text{intexp}} : \langle \text{intexp} \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z}), \quad \llbracket \cdot \rrbracket_{\text{boolexp}} : \langle \text{boolexp} \rangle \rightarrow (\Sigma \rightarrow \mathbb{B}).$$

Hecho clave: las expresiones *siempre terminan* (sin error).

Semántica de comandos

Un comando puede no terminar. Introducimos $\perp$ (*bottom*) para la no-terminación y $\Sigma_{\perp} = \Sigma \cup \{\perp\}$. Entonces

$$\llbracket \cdot \rrbracket_{\text{comm}} : \langle \text{comm} \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp}).$$

Definición 1.2 (Extensión estricta). Si $f : \Sigma \rightarrow \Sigma_{\perp}$, su extensión estricta $f_{\perp} : \Sigma_{\perp} \rightarrow \Sigma_{\perp}$ es $f_{\perp}(\sigma) = \perp$ si $\sigma = \perp$, y $f(\sigma)$ si no.

Ecuaciones semánticas (dirigidas por la sintaxis):

$$\begin{aligned} \llbracket v := e \rrbracket_{\text{comm}} \sigma &= [\sigma \mid v : \llbracket e \rrbracket_{\text{intexp}} \sigma] \\ \llbracket \text{skip} \rrbracket_{\text{comm}} \sigma &= \sigma \\ \llbracket c_0 ; c_1 \rrbracket_{\text{comm}} \sigma &= (\llbracket c_1 \rrbracket_{\text{comm}})_{\perp} (\llbracket c_0 \rrbracket_{\text{comm}} \sigma) \\ \llbracket \text{if } b \text{ then } c_0 \text{ else } c_1 \rrbracket_{\text{comm}} \sigma &= \text{if } \llbracket b \rrbracket_{\text{boolexp}} \sigma \text{ then } \llbracket c_0 \rrbracket_{\text{comm}} \sigma \text{ else } \llbracket c_1 \rrbracket_{\text{comm}} \sigma \end{aligned}$$

Ejemplo resuelto

$$\begin{aligned} \llbracket x := x - 1 ; y := y + x \rrbracket_{\text{comm}} \sigma &= (\llbracket y := y + x \rrbracket_{\text{comm}})_{\perp} (\llbracket x := x - 1 \rrbracket_{\text{comm}} \sigma) \\ &= (\llbracket y := y + x \rrbracket_{\text{comm}})_{\perp} [\sigma \mid x : \sigma x - 1] \quad (\neq \perp, \text{ así que aplica } \llbracket y := y + x \rrbracket_{\text{comm}}) \\ &= [\sigma \mid x : \sigma x - 1 \mid y : \sigma y + (\sigma x - 1)] \\ &= [\sigma \mid x : \sigma x - 1 \mid y : \sigma y + \sigma x - 1]. \end{aligned}$$

Conexiones y trampas

El **while** no se puede definir así. Desenrollando, **while** b **do** $c \equiv \text{if } b \text{ then } (c ; \text{while } b \text{ do } c) \text{ else skip}$, de donde sale la *ecuación de desenrollado*

$$\llbracket \text{while } b \text{ do } c \rrbracket_{\text{comm}} \sigma = \text{if } \llbracket b \rrbracket_{\text{boolexp}} \sigma \text{ then } (\llbracket \text{while } b \text{ do } c \rrbracket_{\text{comm}})_{\perp} (\llbracket c \rrbracket_{\text{comm}} \sigma) \text{ else } \sigma. \quad (2.2)$$

No es una ecuación semántica válida: no está dirigida por la sintaxis (aparece **while** b **do** c a la derecha). Es una *condición*, y puede tener cero, una o varias soluciones.

- **while** $x \neq 0$ **do** $x := x - 2$: para x impar o negativo (no termina) la ecuación deja el resultado libre $\Rightarrow$ varias soluciones.
- **while true do skip**: toda función $\Sigma \rightarrow \Sigma_{\perp}$ la satisface.

Gancho a §2.3: hace falta ordenar los significados por “grado de definición” ($\sqsubseteq$) y tomar el *menor punto fijo* del funcional de desenrollado. Para eso: *dominios y funciones continuas*.

1.2. Dominios y funciones continuas (Reynolds §2.3)

Motivación

El **while** nos dejó la ecuación de desenrollado (2.2): una *condición* con posiblemente muchas soluciones. Necesitamos un criterio para elegir *la* correcta — la que no inventa información. La idea: ordenar los significados por “grado de definición” ($\sqsubseteq$) y quedarnos con el menor. Para que ese menor exista y se comporte bien, montamos la estructura de *dominio*.

Orden, cadenas y límites

Definición 1.3 (Orden de aproximación). $x \sqsubseteq y$ se lee “ x aproxima a y ”, o “ y tiene al menos tanta información como x ”.

Definición 1.4 (Cadena, límite, cadena interesante). Una *cadena* es una sucesión creciente, infinita y numerable $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$ (la igualdad está permitida). Su *límite* es su menor cota superior $\bigsqcup_i x_i$. Una cadena es *interesante* si no contiene a su propio límite o, equivalentemente, si tiene infinitos elementos distintos.

Intuición. $\sqsubseteq$ mide cuánta información tenés; una cadena es un proceso que *gana* información; el límite es la información *total*; “interesante” es el filtro de las cadenas que de verdad ponen a prueba la estructura (las aburridas, eventualmente constantes, contienen su límite trivialmente).

Ejemplo 1.5 (¿Cadena? ¿Interesante? ¿Límite?).

sucesión	¿cadena?	¿interesante?	límite
$0 \sqsubseteq 1 \sqsubseteq 2 \sqsubseteq \dots$ en $\mathbb{N} \cup \{\infty\}$	sí	sí	∞
$5 \sqsubseteq 5 \sqsubseteq \dots$	sí	no	5
$\perp \sqsubseteq \perp \sqsubseteq 7 \sqsubseteq 7 \sqsubseteq \dots$ en $\mathbb{Z}_{\perp}$	sí	no	7
$\{1\} \sqsubseteq \{1, 2\} \sqsubseteq \dots$ (por $\subseteq$)	sí	sí	$\{1, 2, 3, \dots\}$
$3 \sqsupseteq 2 \sqsupseteq 1$ (decreciente)	no	—	—

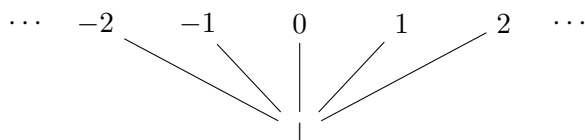
Predominios y dominios

Definición 1.6 (Predominio y dominio). Un orden parcial P es un *predominio* si toda cadena de P tiene su límite en P (la condición solo muerde en las

cadenas interesantes). Un *dominio* es un predominio con elemento mínimo $\perp$, es decir $\perp \sqsubseteq x$ para todo x .

Las dos condiciones son independientes: podés tener $\perp$ sin ser predominio, o ser predominio sin tener $\perp$. Conviene chequearlas por separado.

Lifting y dominios planos. El *lifting* $P_\perp$ se forma agregándole a P un mínimo $\perp$ nuevo, debajo de todo. Convierte cualquier predominio en dominio sin crear cadenas interesantes nuevas. Un *dominio plano* es un conjunto S ordenado discretamente ($x \sqsubseteq y \iff x = y$) al que se le agrega $\perp$: por ejemplo $\mathbb{Z}_\perp = (\mathbb{Z} \text{ discreto}) + \perp$.



Dos niveles y nada en el medio: $\perp$ abajo, una fila chata de enteros incomparables arriba. La información es binaria (o $\perp$, o el valor completo), y por eso un dominio plano **no tiene cadenas interesantes**.

Ejemplo 1.7 (Zoológico). **Dominios:** $\mathbb{Z}_\perp$; $\mathbb{N} \cup \{\infty\}$ con $\leq$ (el más simple *con* cadenas interesantes); $[0, 1]$ con $\leq$.

Predominio pero NO dominio: $\mathbb{Z}$ discreto (sin $\perp$); $(0, 1]$ (no hay mínimo); $\{a, b\}$ discreto.

Ni siquiera predominio: $(\mathbb{N}, \leq)$ (la cadena $0 \sqsubseteq 1 \sqsubseteq \dots$ no tiene cota superior, ¡aunque $\mathbb{N}$ sí tiene mínimo!); $\mathbb{Q} \cap [0, 2]$ (la cadena $1,4 \sqsubseteq 1,41 \sqsubseteq \dots$ apunta a $\sqrt{2} \notin \mathbb{Q}$).

Ejemplo 1.8 (Los intervalos: un punto cambia todo).

conjunto	borde abajo	borde arriba	¿predominio?	¿dominio?
$[0, 1]$	cerrado	cerrado	sí	sí (mín 0)
$(0, 1]$	abierto	cerrado	sí	no (sin mín)
$(0, 1)$	abierto	abierto	no	no
$[0, 1)$	cerrado	abierto	no	no (¡aunque hay mín 0!)

El borde de *arriba* decide predominio; el de *abajo*, si hay mínimo.

Funciones monótonas y continuas

Definición 1.9 (Monótona). $f : D \rightarrow D'$ es *monótona* si $x \sqsubseteq y \Rightarrow f(x) \sqsubseteq f(y)$.

Intuición: más información de entrada nunca produce menos información de salida. (Una f monótona manda cadenas a cadenas.)

Definición 1.10 (Continua). $f : P \rightarrow P'$ es *continua* si preserva los límites de las cadenas: para toda cadena $x_0 \sqsubseteq x_1 \sqsubseteq \dots$,

$$f\left(\bigsqcup_i x_i\right) = \bigsqcup_i f(x_i).$$

Intuición: f no da sorpresas en el límite — está determinada por su comportamiento en las aproximaciones finitas. Lo que f le hace al límite es el límite de lo que f le hace a cada paso.

Proposición 1.11 (Continua $\Rightarrow$ monótona). *Toda función continua es monótona.*

Demostración. Sea f continua y $x \sqsubseteq y$. Consideramos la cadena $x \sqsubseteq y \sqsubseteq y \sqsubseteq \dots$, cuyo límite es y . Por continuidad, $f(y) = \bigsqcup \{f(x), f(y)\}$. Como un lub es, en particular, cota superior, $f(x) \sqsubseteq f(y)$. $\square$

La vuelta es **falsa**: hay funciones monótonas no continuas.

Ejemplo 1.12 (Monótona pero no continua). En $D = \mathbb{N} \cup \{\infty\}$, sobre la cadena $0 \sqsubseteq 1 \sqsubseteq 2 \sqsubseteq \dots$ (límite ∞):

- $g(x) = x + 1$ (con $\infty + 1 = \infty$), de D en D : $g(\bigsqcup x_i) = g(\infty) = \infty$ y $\bigsqcup g(x_i) = \bigsqcup \{1, 2, 3, \dots\} = \infty$. Iguales $\Rightarrow$ **continua** (acompaña al proceso).
- $f(x) = \top'$ si $x = \infty$, y $\perp'$ si x es finito, de D en $\{\perp' \sqsubseteq \top'\}$: $f(\bigsqcup x_i) = f(\infty) = \top'$ pero $\bigsqcup f(x_i) = \bigsqcup \{\perp', \perp', \dots\} = \perp'$. Distintos $\Rightarrow$ **no continua**.

f es monótona pero “se guarda la sorpresa para el final”: salta a $\top'$ recién en ∞ . La continuidad prohíbe exactamente eso.

Conexiones y trampas

- $\perp =$ **no-terminación** / “**no sé nada**”. Análogo de la divergencia; en Rust, el tipo `!` (never).
- **Información binaria en los planos.** En $\mathbb{Z}_\perp$ o $\Sigma_\perp$: o no sabés nada ($\perp$) o lo sabés todo; no existe el “entero a medio saber”. Por eso no hay cadenas interesantes.
- **Las dos palancas de los intervalos.** Borde de arriba $\rightarrow$ predominio; borde de abajo $\rightarrow$ mínimo. Independientes: $[0, 1)$ tiene mínimo pero no es predominio.
- **Cómo chequear predominio.** No alcanza con que “las que se me ocurren” tengan límite: hay que *cazar* la cadena que trepa hacia un borde excluido.

- **Todo poset finito es predominio** (no hay lugar para infinitos distintos); solo falta ver si tiene $\perp$.
- **Continuidad $\approx$ computabilidad.** Una computación honesta solo mira finito de su entrada: no puede “detectar el infinito de un saque”. Por eso los significados son funciones continuas.

Capítulo 2

Cálculo lambda

Capítulo 3

Lenguajes aplicativos e Iswim